Compose, componentes e a CI

DIM0524 — Sistemas para Dispositivos Móveis

Aulas 03–04 (Sprint 0) · 31/08 e 02/09

Roteiro da semana

Segunda, 31/08 — Compose

Parte	O quê
Onde o app se encaixa	O app, a API e o domínio compartilhado
A estrutura do projeto	Os módulos `shared` e `app`, e a regra de dependência
UI declarativa	O que é `@Composable` e recomposição
Estado e layout	`remember`, elevação de estado, `Column` / `Row` / `Box`, preview

Quarta, 02/09 — Componentes, tema e CI

Parte	O quê
Componentes próprios	Reutilizáveis, com *slots*
Material 3	Esquema de cor e tipografia
Kotlin idiomático	Funções de extensão e de escopo
Projeto e CI	Catálogo de versões, `kdoctor`, `ktlint` e `detekt`

Onde paramos

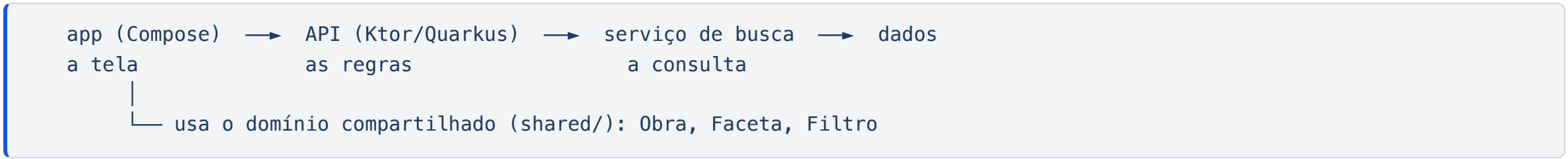
Na aula passada, a linguagem e a plataforma:

```
Kotlin      val/var, null safety, data class, sealed, when
KMP         commonMain, expect/actual, um módulo, vários alvos
Compose     a mesma API para Android, desktop e iOS
```

Hoje usamos isso para construir interface. O domínio — `Obra`, `Faceta`, `Filtro` — já está em `commonMain`; falta a tela que o mostra.

Onde o app se encaixa

O app não vive sozinho: ele é o cliente de um sistema com outras partes.



- Na Sprint 0 vocês escolhem a estratégia de backend (ver `STACK.md`); as telas vêm primeiro
- O mesmo domínio vive no app e na API, o `shared/` é importado pelos dois

A estrutura do projeto

Os dois módulos — e a regra que os separa:

```
shared/     o domínio, Kotlin puro    → commonMain, sem Compose
   │  build.gradle.kts: kotlin { jvm(); /* androidTarget() */ }
   ▼
app/        a interface, em Compose    → depende de :shared
      build.gradle.kts: implementation(project(":shared"))
```

No MUSI, `shared/commonMain` não pode importar Compose, e o compilador garante isso. O alvo `androidTarget()` fica comentado até a sprint onde vamos fazer o deploy para Android ou iOS.

> Observem que a UI conhece o domínio; o domínio não conhece a UI. Esse padrão de projeto permite que as regras de negócio sejam testadas, evoluídas e reaproveitadas sem depender da interface: o mesmo shared/ roda no app (Compose) e no serviço api-ktor, e a tela pode mudar sem mexer no domínio.

Onde estudar depois

Guia	Foco
Compose Multiplatform · `kotlinlang.org/compose-multiplatform`	A API que usamos
Now in Android — Compose · `developer.android.com/compose`	Referência e receitas
Compose Hot Reload · `github.com/JetBrains/compose-hot-reload`	Hot reload no desktop

O material do Android serve: o Compose Multiplatform é a mesma API do Jetpack Compose.

Parte 1

UI declarativa

Duas formas de construir uma tela

Imperativa (Views clássicas)	Declarativa (Compose)
Cria o widget e guarda a referência	Descreve a tela em função do estado
`textView.setText(novo)` a cada mudança	Muda o estado; a UI se redesenha
A tela e os dados podem divergir	A tela é sempre reflexo do estado

```
Imperativa   estado muda → você atualiza cada widget na mão
Declarativa  estado muda → a função roda de novo e redesenha
```

> O bug clássico da forma imperativa é a tela mostrar um valor e os dados terem outro. Na declarativa isso não acontece: a tela é uma função do estado.

`@Composable`: uma função que descreve UI

```
@Composable
fun Saudacao(nome: String) {
    Text("Olá, $nome")
}
```

- Uma função anotada com `@Composable` descreve um pedaço de tela
- Não devolve um objeto de view: emite elementos ao ser chamada
- Só pode ser chamada de dentro de outra função `@Composable`

```
@Composable
fun Tela() {
    Column {
        Saudacao("Ana")
        Saudacao("Bruno")
    }
}
```

> Telas são feitas compondo funções pequenas, como `CartaoObra` dentro de uma lista.

Imperativa x Declarativa

A diferença que separa o Compose das Views imperativas clássicas está no que a função **faz** ao ser chamada.

```
// View clássica: fabrica e devolve um objeto
fun criarSaudacao(nome: String): TextView {
    val tv = TextView(contexto)
    tv.text = "Olá, $nome"
    return tv                       // ← você recebe o widget e o guarda
}

// Compose: não devolve nada; emite ao ser chamada
@Composable
fun Saudacao(nome: String) {
    Text("Olá, $nome")              // ← encaixa um nó Text na posição atual
}
```

- A função `@Composable` retorna `Unit`: não há objeto de view para guardar
- **Emitir** é registrar um nó na árvore de UI que o Compose está montando

> Chamar `Saudacao("Ana")` não te dá um `Text` — coloca um `Text` ali. A tela é efeito da chamada, não o valor de retorno.

Aninhar é montar a árvore

Como cada chamada emite na posição atual, o aninhamento das chamadas vira a estrutura da tela:

```
@Composable
fun Tela() {
    Column {
        Saudacao("Ana")
        Saudacao("Bruno")
    }
}
```

```
  Column
  ├── Text  "Olá, Ana"
  └── Text  "Olá, Bruno"
```

Quem sabe "onde estou montando" é um parâmetro invisível — o `Composer` — que o compilador injeta em toda função `@Composable`.

> Por isso **uma `@Composable` só pode ser chamada de dentro de outra**: ela precisa do `Composer` em escopo para saber onde encaixar o elemento.

Imperativa x Declarativa

Como você nunca recebe o nó, não dá para atualizar a tela "por fora":

```
val t = Text("Olá")      // não há retorno: Text devolve Unit
t.setText("Oi")          // não há objeto nem setter para chamar
```

Para mudar o que aparece, muda-se o **estado** que a função lê — e o Compose chama a função de novo (recompõe).

Views clássicas	Compose
Guarda a referência do widget	Não há referência para guardar
`widget.setX(novo)` atualiza	Muda o estado; a função reemite
Tela e dados podem divergir	Tela é sempre função do estado

> No Compose você não conserta a tela, você a **descreve de novo**.

Recomposição

Quando o estado que uma função lê muda, o Compose chama a função de novo — só ela, não a tela inteira. Isso é a recomposição.

```
estado: filtro = "baião"          →  a lista recompõe, mostra 2 obras
usuário seleciona "ijexá"         →  filtro = "ijexá"  →  recompõe, mostra 1
```

Consequência	O que significa para você
A função pode rodar muitas vezes	Não adicione efeitos colaterais
A ordem de execução não é garantida	Não dependa de "rodou antes/depois"
Deve ser rápida e sem I/O	Ler arquivo ou rede aqui trava a tela

> Pense na função como uma descrição, não como um passo a passo. O Compose decide quando executá-la.

Parte 2

Estado

O que faz a tela mudar.

O problema: variável comum não sobrevive

```
@Composable
fun Contador() {
    var n = 0                        // ← recriada a cada recomposição
    Button(onClick = { n++ }) {
        Text("Cliques: $n")
    }
}
```

Isso não funciona: a cada recomposição `n` volta a zero, e o Compose nem sabe que precisa recompor.

Faltam duas coisas:

1. Lembrar o valor entre recomposições
2. Avisar o Compose quando ele muda

`remember` + `mutableStateOf`

```
@Composable
fun Contador() {
    var n by remember { mutableStateOf(0) }
    Button(onClick = { n++ }) {
        Text("Cliques: $n")
    }
}
```

Peça	Papel
`mutableStateOf(0)`	Um estado observável: mudou, recompõe quem o lê
`remember { ... }`	Preserva o valor entre recomposições
`by`	Para ler/escrever `n` direto, sem `.value`

`remember` sem `mutableStateOf` guarda mas não avisa; `mutableStateOf` sem `remember` avisa mas esquece. Os dois juntos são o padrão.

Anatomia de `var n by remember { … }`

O jeito de entender a linha é **tirar cada peça** e ver o que quebra:

```
var n by remember { mutableStateOf(0) }
```

Sem esta peça	O que acontece na tela
`mutableStateOf`	muda o valor, mas **não redesenha** — nada avisa o Compose
`remember`	a tela **esquece**: o valor volta a 0 a cada recomposição
`by`	funciona igual — só que com `.value` em todo lugar

- `mutableStateOf` faz a tela **reagir**, `remember` faz ela **lembrar**, `by` some com o `.value`
- É `var`, não `val`: `n` é lido **e** escrito (`n++`)

> O `by` exige dois imports de `androidx.compose.runtime` — `getValue` e `setValue`. Sem eles, o erro é *"getValue is not found"* na linha do delegate. Vale avisar antes.

`remember` com chave

`remember(chave)` recalcula quando a chave muda. É como derivar um valor sem recomputar à toa:

```
val visiveis = remember(obras, filtro) {
    if (filtro == null) obras
    else obras.filter { it.satisfaz(filtro) }
}
```

Enquanto `obras` e `filtro` não mudarem, a lista filtrada é reaproveitada. Quando um deles muda, o bloco roda de novo.

> É deste `TelaAcervo` do projeto de exemplo. Repare que a filtragem é derivada do estado, não um estado novo guardado à parte — menos coisa para manter em sincronia.

Parte 3

Elevação de estado

Guardar estado dentro nem sempre serve

```
@Composable
fun TelaAcervo(obras: List<Obra>) {
    var filtro by remember { mutableStateOf<Filtro?>(null) }   // preso aqui
    // ...
}
```

Funciona — mas ninguém de fora consegue decidir o que a tela mostra, testar com um filtro fixo, ou pré-visualizar estados diferentes.

O estado ficou preso dentro do componente.

Elevar o estado (*state hoisting*)

Suba o estado para quem chama, e receba valor mais evento:

```
@Composable
fun TelaAcervo(
    obras: List<Obra>,
    filtro: Filtro?,                    // o que mostrar (valor)
    aoTrocarFiltro: (Filtro?) -> Unit,  // o que fazer ao mudar (evento)
    modifier: Modifier = Modifier,
) { /* ... */ }
```

```
  estado desce  ──────►  filtro
  evento sobe   ◄──────  aoTrocarFiltro(novo)
```

Ganho	Por quê
Testável	Passe um filtro e verifique a saída
Pré-visualizável	Cada estado vira um `@Preview`
Reutilizável	O componente não decide nada sozinho

> Estado desce, evento sobe. Um componente sem estado interno é o que a rubrica chama de próprio e reutilizável.

Componente próprio: `CartaoObra`

Não conhece a tela em que está, não guarda estado, não decide nada sobre o acervo. Recebe uma obra e a desenha:

```
@Composable
fun CartaoObra(obra: Obra, modifier: Modifier = Modifier) {
    Card(modifier.fillMaxWidth()) {
        Column(Modifier.padding(12.dp)) {
            Text(obra.titulo, style = MaterialTheme.typography.titleMedium)
            Text("${obra.artista} · ${obra.ano}",
                 style = MaterialTheme.typography.bodyMedium)
        }
    }
}
```

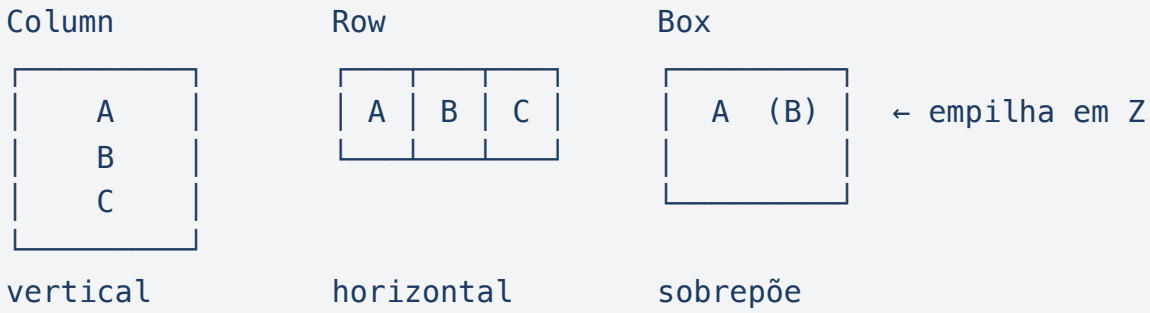
Serve na lista, numa tela de detalhe ou num preview. É o entregável da Sprint 0.

> Convenção: todo componente aceita `modifier: Modifier = Modifier` como último parâmetro. Já vemos por quê.

Parte 4

Layout e modificadores

Os três contêineres




```
Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
    Text("Acervo")
    Text("5 obras")
}
```

Contêiner	Empilha	Alinha com
`Column`	de cima para baixo	`horizontalAlignment`, `verticalArrangement`
`Row`	da esquerda para a direita	`verticalAlignment`, `horizontalArrangement`
`Box`	em profundidade	`contentAlignment`

`Modifier`: tamanho, espaço, aparência, gesto

Um `Modifier` ajusta como um componente aparece e se comporta. A ordem importa — ela é aplicada de cima para baixo:

```
Text(
    "Acervo",
    modifier = Modifier
        .fillMaxWidth()      // ocupa a largura
        .padding(16.dp)      // espaço interno
)
```

Modificador	Efeito
`.fillMaxWidth()` · `.fillMaxSize()`	Ocupa o espaço disponível
`.padding(16.dp)`	Espaço ao redor
`.size(48.dp)`	Tamanho fixo
`.clickable { ... }`	Reage ao toque

`.padding(8.dp).background(azul)` é diferente de `.background(azul).padding(8.dp)`. Trocar a ordem troca o resultado. Testem os dois no Hot Reload.

Por que o `modifier` vem de fora

O componente não decide seu próprio espaçamento externo. Quem o usa decide:

```
@Composable
fun CartaoObra(obra: Obra, modifier: Modifier = Modifier) {
    Card(modifier.fillMaxWidth()) { /* ... */ }
}

// quem chama posiciona:
CartaoObra(obra, Modifier.padding(8.dp))
```

É a mesma ideia da elevação de estado, aplicada ao layout: o componente descreve o que é; onde ele fica é decisão de quem o coloca na tela.

> Um componente que fixa a própria margem externa não serve em dois lugares diferentes. Deixe o `modifier` entrar por parâmetro.

Parte 5

Preview e Hot Reload

`@Preview`: ver sem rodar o app

Porque o componente tem estado elevado, cada estado vira uma pré-visualização:

```
@Preview
@Composable
fun CartaoObraPreview() {
    CartaoObra(
        Obra("obra-01", "Ponteio", "Edu Lobo", 1967, emptyList())
    )
}

@Preview
@Composable
fun ListaVaziaPreview() {
    TelaAcervo(obras = emptyList(), filtro = null, aoTrocarFiltro = {})
}
```

> Aqui aparece o retorno do estado elevado: dá para pré-visualizar a lista vazia e a lista cheia sem tocar no app. Componente com estado preso não permite isso.

Hot Reload: o ciclo rápido

No alvo desktop, o hot reload aplica a mudança do código na hora, mantendo o estado da tela:

```
edita o CartaoObra  →  salva  →  a tela atualiza em ~1 s
                                  sem reiniciar, sem emulador
```

Ciclo	Custo
Emulador Android	dezenas de segundos por mudança
Desktop + Hot Reload	cerca de 1 segundo

> No Codespaces: abra a porta 6080 (noVNC) e rode `DISPLAY=:1 ./gradlew :app:hotRunJvm -Pheadless` para recarregar ao salvar — ou `:app:run -Pheadless` só para ver a tela. Passo a passo em docs/COMO-RODAR.md.

Parte 6

Mãos à obra

Oficina — 25 minutos `alvo desktop`

Partindo do esqueleto KMP, com o `acervoDeExemplo` já pronto:

1. Escreva CartaoObra(obra): título, artista · ano
2. Monte TelaAcervo(obras) com uma LazyColumn de CartaoObra
3. Eleve o filtro: receba `filtro` e `aoTrocarFiltro` por parâmetro
4. Adicione FiltrosRapidos com FilterChip (todos / baião / até 1970)
5. Crie dois @Preview: lista cheia e lista vazia

```
LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
    items(visiveis, key = { it.id }) { obra -> CartaoObra(obra) }
}
```

`LazyColumn` só compõe o que aparece na tela — é a lista que aguenta mil itens. `key` ajuda o Compose a reaproveitar os cartões ao rolar.

Erros que aparecem sempre

Erro	O que acontece
`var n = 0` sem `remember`	Volta a zero a cada recomposição
`remember` sem `mutableStateOf`	Guarda, mas a tela não recompõe
I/O ou log no corpo do `@Composable`	Roda muitas vezes, trava a tela
Estado preso dentro do componente	Não dá para testar nem pré-visualizar
`Column` gigante em vez de `LazyColumn`	Compõe tudo de uma vez; trava com lista grande
Cor crua em vez do tema	Ignora modo claro e escuro

> Os dois primeiros respondem por quase toda dúvida de "por que não atualiza / por que zera". Guardem a dupla `remember { mutableStateOf(...) }`.

Quarta · 02/09

Componentes, tema e CI

Componente próprio, de novo

Na segunda, `CartaoObra` já era um componente próprio: recebe uma obra, não guarda estado, serve na lista e no preview.

```
@Composable
fun CartaoObra(obra: Obra, modifier: Modifier = Modifier) { … }
```

Hoje damos o próximo passo: componentes que recebem conteúdo de fora — os *slots*.

> Abram `app/src/commonMain/kotlin/br/ufrn/musi/ui/TelaAcervo.kt`: `CartaoObra` e `FiltrosRapidos` são os dois componentes próprios que a Sprint 0 pede.

Slots: o conteúdo vem de fora

Um *slot* é um parâmetro `@Composable`. O componente decide o arranjo; quem chama decide o conteúdo.

```
@Composable
fun Secao(titulo: String, conteudo: @Composable () -> Unit) {
    Column(Modifier.padding(12.dp)) {
        Text(titulo, style = MaterialTheme.typography.titleMedium)
        conteudo()                          // ← o slot
    }
}

Secao("Acervo") {
    LazyColumn { items(obras) { CartaoObra(it) } }
}
```

O `Card { … }` que vocês usaram na segunda já é assim: ele desenha a moldura e recebe o conteúdo por um slot. É o padrão mais comum do Compose.

Material 3: cor e tipografia do tema

Nunca escreva cor ou tamanho cru. Use os papéis do tema:

```
Text(obra.titulo,  style = MaterialTheme.typography.titleMedium)
Text(obra.artista, color = MaterialTheme.colorScheme.onSurfaceVariant)
```

Em vez de	Use
`fontSize = 20.sp`	`typography.titleMedium`
`Color(0xFF666666)`	`colorScheme.onSurfaceVariant`

> É o que `CartaoObra` faz no MUSI. Usar papéis do tema é o que faz o app respeitar modo claro e escuro sem retrabalho.

O tema num lugar só

`MaterialTheme` embrulha a árvore e injeta cor, tipografia e formas. Trocar o esquema muda o app inteiro:

```
MaterialTheme(colorScheme = meuEsquema) {
    TelaAcervo(obras = acervoDeExemplo, filtro = filtro, aoTrocarFiltro = { … })
}
```

> Definam o esquema uma vez, na raiz. Componente que fixa a própria cor quebra o modo escuro e não dá para reaproveitar noutro tema.

Kotlin idiomático: funções de escopo

Cinco funções curtas que deixam o código de UI mais limpo:

Função	Devolve	Uso típico
`let`	o resultado do bloco	agir sobre algo não-nulo
`apply`	o próprio objeto	configurar um objeto
`also`	o próprio objeto	efeito colateral (log)
`run` / `with`	o resultado do bloco	agrupar chamadas

```
val texto = filtro?.let { descrever(it) } ?: "sem filtro"
```

Esse `let` é do `TelaAcervo` do MUSI: só chama `descrever` quando `filtro` não é nulo, e cai no `?:` quando é. Sem `if` e sem variável temporária.

Gradle e o catálogo de versões

Uma fonte da verdade para todas as versões: `gradle/libs.versions.toml`.

```
[versions]
kotlin = "2.4.10"
compose = "1.12.0"

[libraries]
koin-core = { module = "io.insert-koin:koin-core", version.ref = "koin" }

[plugins]
compose-multiplatform = { id = "org.jetbrains.compose", version.ref = "compose" }
```

```
// nos build.gradle.kts:
plugins { alias(libs.plugins.compose.multiplatform) }
dependencies { implementation(libs.koin.core) }
```

> Versão duplicada em vários módulos é fonte garantida de conflito. O catálogo mantém tudo num arquivo só — é o `libs.versions.toml` do MUSI.

CI: agora com `ktlint` e `detekt`

O workflow deixa de só compilar e passa a exigir estilo e qualidade:

```
push / PR ⟶ compila ⟶ ktlintCheck ⟶ detekt ⟶ verde
```

Ferramenta	Pega
ktlint	Formatação fora do padrão Kotlin
detekt	Complexidade, *code smells*, funções longas

No MUSI, o job `kotlin` do `.github/workflows/ci.yml` já roda os testes; ktlint e detekt entram a partir da Sprint 1.

Exercício

Partindo da tela da segunda:

> 1. Extraia uma Secao(titulo) { … } com slot, e use no TelaAcervo
> 2. Troque cores/tamanhos crus por MaterialTheme.typography/colorScheme

As tarefas da Sprint 0

O enunciado de cada entrega está em `docs/SPRINT-0-TAREFAS.md`.

Ajuste recente	Detalhe
Backlog: mínimo 5 histórias (antes 10)	≥ 3 estimadas, todas priorizadas
`docs/proposta.md`: até 5 páginas (antes 3)	as 7 seções do guia

> Criem uma issue por tarefa. O guia completo, com templates, continua em `docs/SPRINT-0.md`.

Próximos passos

Até a entrega (11/09)

- App compilando em desktop, com a primeira tela
- Componente próprio e reutilizável, estado elevado, ao menos dois `@Preview`
- `ktlintCheck` e `detekt` passando no GitHub Actions
- Proposta com plataforma-alvo e backend justificados, e o backlog (≥ 5 histórias)

> 09/09 é encontro online, no horário da aula, para dúvidas sobre o ambiente e a proposta.

Referências da semana

Compose e componentes

- *Thinking in Compose* · `developer.android.com/develop/ui/compose/mental-model`
- *State in Compose* · `developer.android.com/develop/ui/compose/state`
- *Compose layout* e *slots* · `developer.android.com/develop/ui/compose/layouts`

Tema, Kotlin e ferramentas

- Material 3 · `m3.material.io` · Compose Multiplatform · `kotlinlang.org/compose-multiplatform`
- Funções de escopo · `kotlinlang.org/docs/scope-functions.html`
- Catálogo de versões · `docs.gradle.org`

Disciplina e projeto de exemplo

- `github.com/fmarquesfilho/sistemas-moveis-2026-2`
- MUSI · `github.com/fmarquesfilho/musi` — `app/`, `shared/`, `gradle/libs.versions.toml`